
Lab Assignment 15.2

Program : B. Tech (CSE)
Course Title : AI Assisted Coding
Course Code : 23CS002PC304
Semester : III
Academic Session : 2025-2026
Name of Student : Ganesh
Enrollment No. : 2403A51L55
Batch No. : 52
Date : 17-03-2026

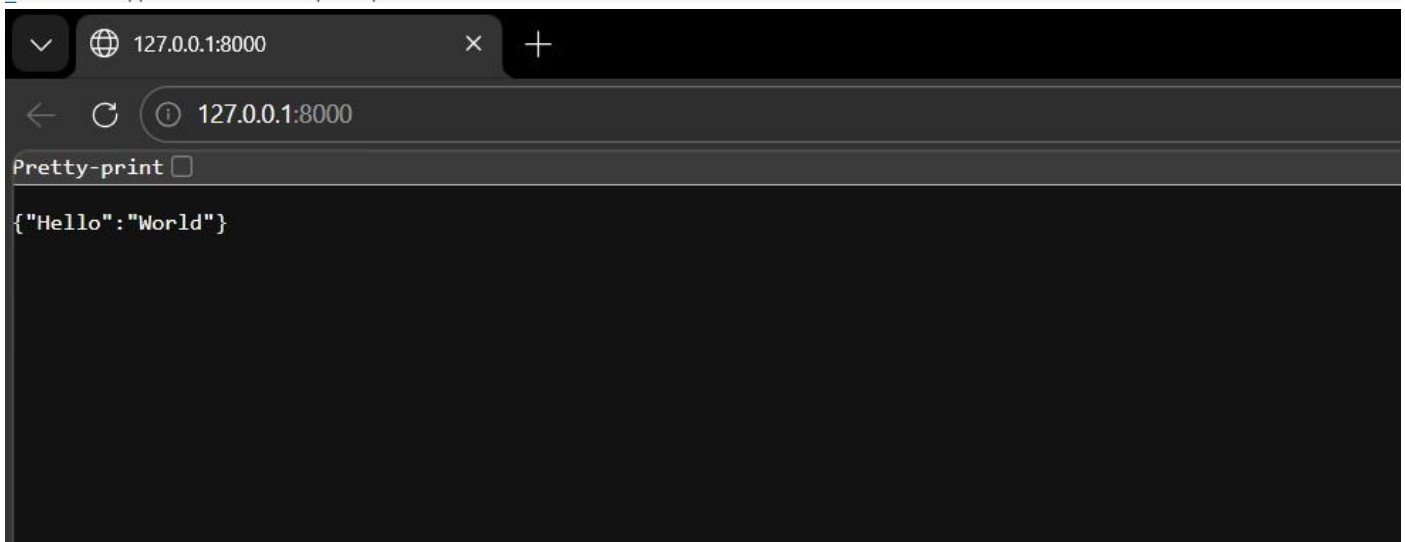
Task 1:

Prompt: Generate a basic Fast API application with one endpoint **Code:**

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
    return {"Hello": "World"}
```

Output:

```
(.venv) PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> python -m uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\BURRA SRUTHI\\OneDrive\\Desktop\\3_2\\AI\\AI_code']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [41568] using StatReload
INFO: Started server process [12964]
INFO: Waiting for application startup.
INFO: Application startup complete.
```



Explanation:

This code creates a basic FastAPI application with one endpoint. The endpoint is defined using the `@app.get("/")` decorator, which means it will respond to GET requests at the root URL ("/"). When a request is made to this endpoint, the `read_root` function is called, which returns a JSON response with the message "Hello": "World".

Task 2:

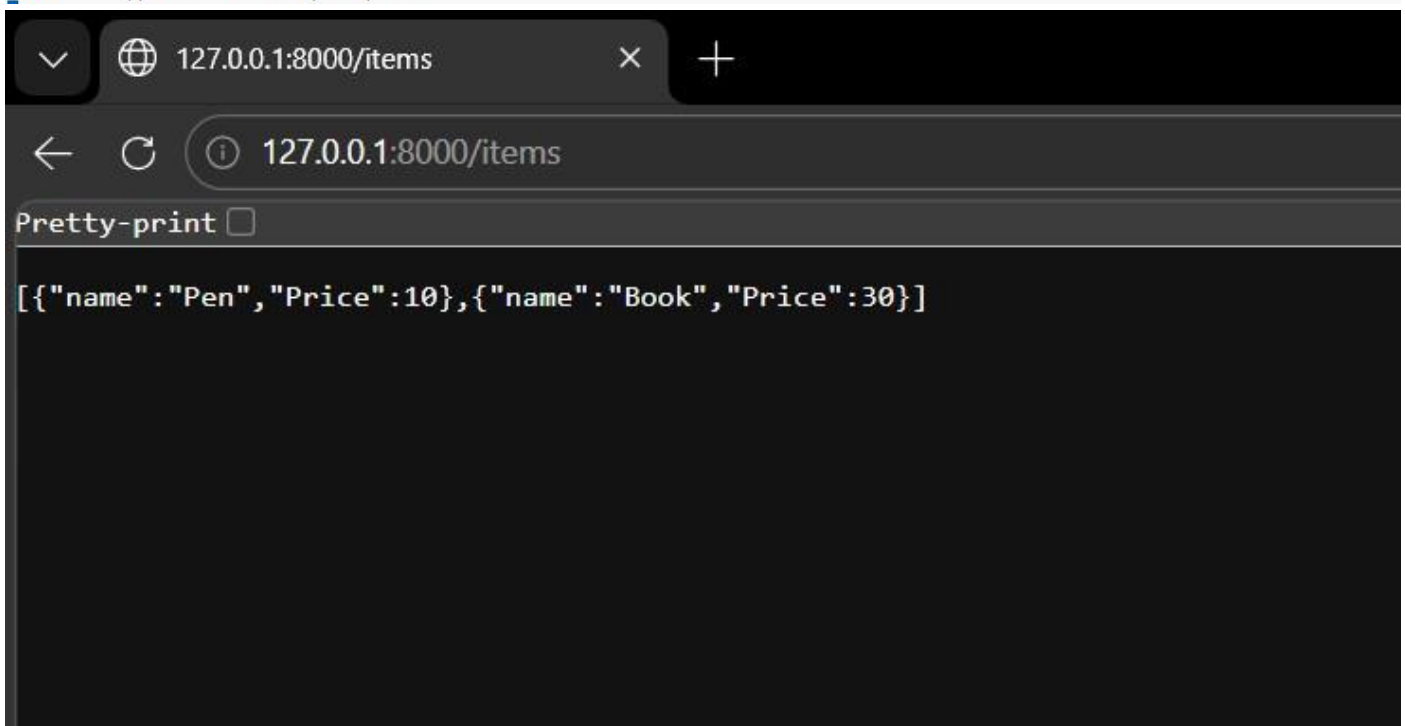
Prompt: Use a python list as temporary storage and Implement endpoints to add and retrieve products.

Code:

```
from fastapi import FastAPI
app = FastAPI()
items = []
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```

Output:

```
(.venv) PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> python -m uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\BURRA SRUTHI\\OneDrive\\Desktop\\3_2\\AI\\AI_code']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [31776] using StatReload
INFO: Started server process [33664]
INFO: Waiting for application startup.
INFO: Application startup complete.
```



Explanation:

This code creates a FastAPI application with two endpoints. The first endpoint (GET /items) retrieves the list of items stored in the 'items' list, while the second endpoint (POST /items) allows you to add a new item to the list by sending a JSON object in the request body. The added item is then returned in the response along with a success message.

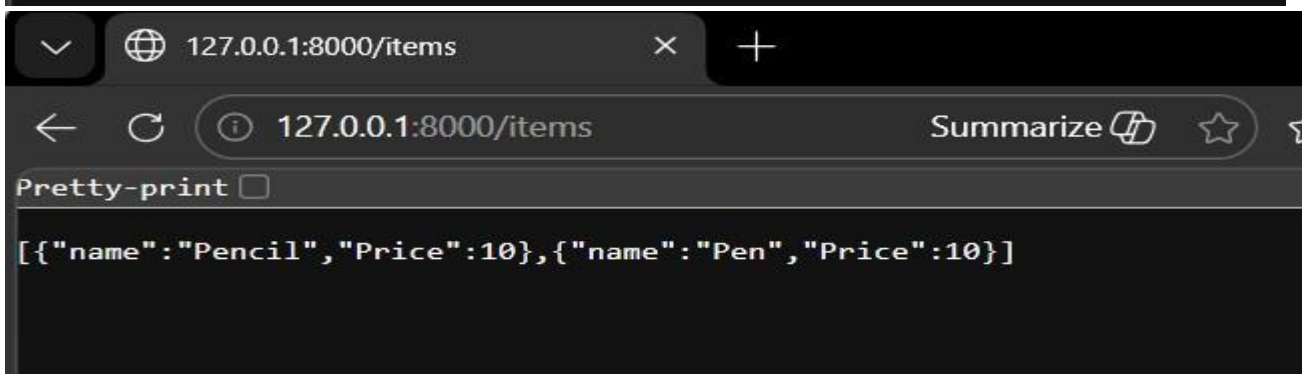
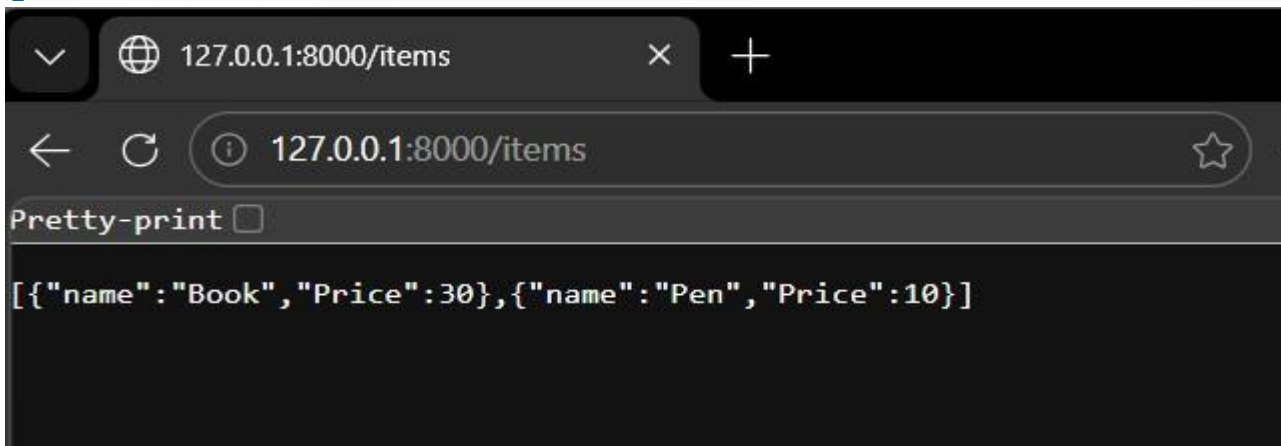
Task 3:

Prompt: Generate an endpoint to update an item using its index. **Code:**

```
from fastapi import FastAPI
app = FastAPI()
items = []
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index < 0 or index >= len(items):
        return {"message": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
```

Output:

```
(.venv) PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> python -m uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\BURRA SRUTHI\\OneDrive\\Desktop\\3_2\\AI\\AI_code']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [35956] using StatReload
INFO: Started server process [20824]
INFO: Waiting for application startup.
INFO: Application startup complete.
```



Explanation:

PUT: This endpoint allows you to update an existing item in the `items` list using its index. The index is provided as a path parameter, and the new item data is expected to be a dictionary in the request body. If the index is valid, the item at that index is updated with the new data, and a success message along with the updated item is returned. If the index is out of bounds, a message indicating that the item was not found is returned.

Task 4:

Prompt: Create a role-based web application that as Admin, Editor, User roles and conditional UI rendering and Design access control logic and maintain secure state transitions and Create scalable UI components. **Code:**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Role-Based Web Application</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 40px; }
    .container { max-width: 500px; margin: auto; }
    .card { border: 1px solid #ccc; border-radius: 8px; padding: 24px; margin-bottom: 16px; }
    .admin { background: #ffe0e0; }
    .editor { background: #e0ffff; }
    .user { background: #e0ffe0; }
    button { margin: 4px; }
    .logout { float: right; }
  </style>
</head>
<body>
  <div class="container" id="app"></div>
  <script>
    // Roles and permissions
    const ROLES = ['Admin', 'Editor', 'User'];
    const PERMISSIONS = {
      Admin: ['manage_users', 'edit_content', 'view_content'],
      Editor: ['edit_content', 'view_content'],
      User: ['view_content']
    };

    // Simulated users
    const USERS = [
      { username: 'admin', password: 'admin123', role: 'Admin' },
      { username: 'editor', password: 'editor123', role: 'Editor' },
      { username: 'user', password: 'user123', role: 'User' }
    ];

    // Secure state management
    let currentUser = null;

    // UI Components
    function LoginForm() {
      return `
        <div class="card">
          <h2>Login</h2>
          <form id="loginForm">
            <label>Username:<br><input type="text" name="username" required></label><br><br>
            <label>Password:<br><input type="password" name="password" required></label><br><br>
            <button type="submit">Login</button>
          </form>
        </div>
      `;
    }

    function Navbar(user) {
      return `
        <div style="margin-bottom:16px;">
          <strong>Role:</strong> ${user.role}
          <button class="logout" id="logoutBtn">Logout</button>
        </div>
      `;
    }
  </script>
</body>
</html>
```



```

    </div>
  };
}

function AdminPanel() {
  return `
    <div class="card admin">
      <h3>Admin Panel</h3>
      <p>Manage users and system settings.</p>
      <button onclick="alert('User management accessed!')">Manage Users</button>
    </div>
  `;
}

function EditorPanel() {
  return `
    <div class="card editor">
      <h3>Editor Panel</h3>
      <p>Edit and manage content.</p>
      <button onclick="alert('Content editor accessed!')">Edit Content</button>
    </div>
  `;
}

function UserPanel() {
  return `
    <div class="card user">
      <h3>User Dashboard</h3>
      <p>View content and updates.</p>
      <button onclick="alert('Viewing content!')">View Content</button>
    </div>
  `;
}

// Access control logic
function hasPermission(role, permission) {
  return PERMISSIONS[role] && PERMISSIONS[role].includes(permission);
}

// Main render function
function render() {
  const app = document.getElementById('app');
  if (!currentUser) {
    app.innerHTML = LoginForm();
    document.getElementById('loginForm').onsubmit = function(e) {
      e.preventDefault();
      const username = this.username.value;
      const password = this.password.value;
      const user = USERS.find(u => u.username === username && u.password === password);
      if (user) {
        currentUser = { ...user }; // Secure state transition
        render();
      } else {
        alert('Invalid credentials!');
      }
    };
  } else {
    let panels = '';
    if (hasPermission(currentUser.role, 'manage_users')) panels += AdminPanel();
    if (hasPermission(currentUser.role, 'edit_content')) panels += EditorPanel();
    if (hasPermission(currentUser.role, 'view_content')) panels += UserPanel();

    app.innerHTML = Navbar(currentUser) + panels;
    document.getElementById('logoutBtn').onclick = function() {
      currentUser = null; // Secure state transition
      render();
    };
  }
}

// Initial render
render();
</script>
</body>
</html>

```

Output:

Login

Username:

Password:

Role: Admin

Admin Panel

Manage users and system settings.

Editor Panel

Edit and manage content.

User Dashboard

View content and updates.

Explanation:

This code implements a simple role-based web application using HTML, CSS, and JavaScript. It defines three user roles: Admin, Editor, and User, each with specific permissions. The application includes a login form, a navigation bar displaying the user's role, and different panels for each role. The access control logic ensures that users can only access features corresponding to their roles. The state management is handled securely by maintaining the current user in a variable and updating the UI accordingly.

Task 5:

Prompt: Design a multilingual website that Switch languages dynamically and persist language preference and structure translation files and it handle RTL/LTR layouts and improve accessibility.

Code:

```
<!DOCTYPE html>
<html lang="en" dir="ltr">
<head>
  <meta charset="UTF-8">
  <title>Multilingual Website</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <style>
    body[dir="rtl"] {
      direction: rtl;
      text-align: right;
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    }
    body[dir="ltr"] {
      direction: ltr;
      text-align: left;
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    }
    .lang-switcher {
      margin: 1em 0;
    }
    .lang-switcher button {
      margin-right: 0.5em;
      padding: 0.5em 1em;
      cursor: pointer;
    }
    main {
      max-width: 600px;
      margin: auto;
    }
    /* Accessibility: focus styles */
    button:focus, a:focus {
      outline: 2px solid #0078D4;
      outline-offset: 2px;
    }
  </style>
</head>
<body>
  <nav aria-label="Language Switcher" class="lang-switcher">
    <button type="button" data-lang="en" aria-label="Switch to English">English</button>
    <button type="button" data-lang="es" aria-label="Switch to Spanish">Español</button>
    <button type="button" data-lang="ar" aria-label="Switch to Arabic">العربية</button>
  </nav>
  <main>
    <h1 id="title"></h1>
    <p id="welcome"></p>
    <a href="#" id="learnMore"></a>
  </main>
  <script>
    // Translation files structure
    const translations = {
      en: {
        dir: "ltr",
        title: "Welcome to Our Website",
        welcome: "This is a multilingual website. Switch languages using the buttons above.",
        learnMore: "Learn more about accessibility"
      }
    }
  </script>
</body>
</html>
```

```

    },
    es: {
      dir: "ltr",
      title: "Bienvenido a Nuestro Sitio Web",
      welcome: "Este es un sitio web multilingüe. Cambia de idioma usando los botones de arriba.",
      learnMore: "Aprende más sobre accesibilidad"
    },
    ar: {
      dir: "rtl",
      title: "مرحبًا بكم في موقعنا",
      welcome: "هذا موقع متعدد اللغات، قم بتغيير اللغة باستخدام الأزرار أعلاه.",
      learnMore: "تعرف على المزيد حول إمكانية الوصول"
    }
  };

  // Accessibility: focus management
  function focusMain() {
    document.querySelector('main').setAttribute('tabindex', '-1');
    document.querySelector('main').focus();
  }

  // Load language from localStorage or default to English
  function getSavedLang() {
    return localStorage.getItem('lang') || 'en';
  }

  function setLang(lang) {
    if (!translations[lang]) lang = 'en';
    localStorage.setItem('lang', lang);
    document.documentElement.lang = lang;
    document.body.setAttribute('dir', translations[lang].dir);

    // Update content
    document.getElementById('title').textContent = translations[lang].title;
    document.getElementById('welcome').textContent = translations[lang].welcome;
    document.getElementById('learnMore').textContent = translations[lang].learnMore;
    document.getElementById('learnMore').setAttribute('href', lang === 'ar' ? 'https://www.w3.org/WAI/standards-guidelines/ar/' : 'https://www.w3.org/WAI/');
  }

  // Event listeners for language switcher
  document.querySelectorAll('.lang-switcher button').forEach(btn => {
    btn.addEventListener('click', e => {
      const lang = btn.getAttribute('data-lang');
      setLang(lang);
      focusMain();
    });
  });

  // Initial load
  setLang(getSavedLang());
</script>
</body>
</html>

```

Output:

Welcome to Our Website

This is a multilingual website. Switch languages using the buttons above.

[Learn more about accessibility](#)

Explanation:

This HTML document is a multilingual website that supports English, Spanish, and Arabic. It includes a language switcher that allows users to change the language of the content dynamically.

Task 6:

Prompt: Create an online exam interface that contains Timer, questions, and submission and prevent multiple submissions (accept only one submission) and manage exam state and improve exam UX and ensure accessibility.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Online Exam Interface</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <style>
    body { font-family: Arial, sans-serif; margin: 0; background: #f9f9f9; }
    .container { max-width: 600px; margin: 40px auto; background: #fff; padding: 24px; border-radius: 8px; box-shadow: 0 2px 8px rgba(0,0,0,0.1); }
    h1 { text-align: center; }
    .timer { font-size: 1.5em; color: #d32f2f; text-align: right; margin-bottom: 16px; }
    fieldset { border: none; margin-bottom: 20px; }
    legend { font-weight: bold; }
    label { display: block; margin: 8px 0; }
    button { background: #1976d2; color: #fff; border: none; padding: 12px 24px; border-radius: 4px; font-size: 1em; cursor: pointer; }
    button[disabled] { background: #aaa; cursor: not-allowed; }
    .submitted-msg { color: #388e3c; font-weight: bold; text-align: center; margin-top: 20px; }
    .visually-hidden { position: absolute; left: -10000px; top: auto; width: 1px; height: 1px; overflow: hidden; }
    @media (max-width: 600px) {
      .container { padding: 10px; }
    }
  </style>
</head>
<body>
  <main class="container" aria-labelledby="exam-title">
    <h1 id="exam-title">Online Exam</h1>
    <div class="timer" id="timer" aria-live="polite" aria-atomic="true">Time Left: 10:00</div>
    <form id="examForm" autocomplete="off" aria-describedby="exam-desc">
      <p id="exam-desc" class="visually-hidden">Answer all questions and submit before the timer ends. Only one submission is allowed.</p>
      <fieldset>
        <legend>1. What is the capital of France?</legend>
        <label>
          <input type="radio" name="q1" value="Paris" required>
          Paris
        </label>
        <label>
          <input type="radio" name="q1" value="London">
          London
        </label>
        <label>
          <input type="radio" name="q1" value="Berlin">
          Berlin
        </label>
      </fieldset>
      <fieldset>
        <legend>2. Which language runs in a web browser?</legend>
        <label>
          <input type="radio" name="q2" value="Java" required>
          Java
        </label>
        <label>
          <input type="radio" name="q2" value="C">
          C
        </label>
        <label>
          <input type="radio" name="q2" value="JavaScript">
          JavaScript
        </label>
      </fieldset>
      <fieldset>
        <legend>3. What does HTML stand for?</legend>
        <label>
          <input type="radio" name="q3" value="Hyper Text Markup Language" required>
          Hyper Text Markup Language
        </label>
        <label>
          <input type="radio" name="q3" value="Home Tool Markup Language">
          Home Tool Markup Language
        </label>
        <label>
          <input type="radio" name="q3" value="Hyperlinks and Text Markup Language">
          Hyperlinks and Text Markup Language
        </label>
      </fieldset>
      <button type="submit" id="submitBtn">Submit Exam</button>
    </form>
    <div id="submittedMsg" class="submitted-msg" role="status" aria-live="polite" style="display:none;">
      Your exam has been submitted. Thank you!
    </div>
  </main>
<script>
  // Exam State
  let submitted = false;
  let timeLeft = 600; // 10 minutes in seconds
  const timerEl = document.getElementById('timer');
  const form = document.getElementById('examForm');
  const submitBtn = document.getElementById('submitBtn');
  const submittedMsg = document.getElementById('submittedMsg');

  // Accessibility: focus management
  function focusFirstInput() {
    const firstInput = form.querySelector('input[type="radio"]');
    if (firstInput) firstInput.focus();
  }

  // Timer logic
  function updateTimer() {
    const min = String(Math.floor(timeLeft / 60)).padStart(2, '0');
    const sec = String(timeLeft % 60).padStart(2, '0');
    timerEl.textContent = `Time Left: ${min}:${sec}`;
    if (timeLeft <= 0) {
      timerEl.textContent = "Time's up!";
      if (!submitted) {
        handleSubmit();
      }
    }
  }
</script>
```

```

    }
}

function startTimer() {
    updateTimer();
    const interval = setInterval(() => {
        if (timeLeft > 0 && !submitted) {
            timeLeft--;
            updateTimer();
        } else {
            clearInterval(interval);
        }
    }, 1000);
}

// Prevent multiple submissions
function handleSubmit(e) {
    if (e) e.preventDefault();
    if (submitted) return;
    submitted = true;
    submitBtn.disabled = true;
    Array.from(form.elements).forEach(el => el.disabled = true);
    submittedMsg.style.display = 'block';
    form.setAttribute('aria-disabled', 'true');
    // Optionally, send data to server here
}

form.addEventListener('submit', handleSubmit);

// Prevent resubmission on reload (optional, for demo)
window.addEventListener('beforeunload', function (e) {
    if (!submitted && timeLeft > 0) {
        e.preventDefault();
        e.returnValue = '';
    }
});

// Accessibility: focus first input on load
window.onload = function() {
    focusFirstInput();
    startTimer();
};
</script>
</body>
</html>

```

Output:

Online Exam

Time Left: 09:55

1. What is the capital of France?

- ☐ Paris
- ☐ London
- ☐ Berlin

2. Which language runs in a web browser?

- ☐ Java
- ☐ C
- ☐ JavaScript

3. What does HTML stand for?

- ☐ Hyper Text Markup Language
- ☐ Home Tool Markup Language
- ☐ Hyperlinks and Text Markup Language

Submit Exam

Online Exam

Time Left: 09:40

1. What is the capital of France?

- ☐ Paris
- ☐ London
- ☒ Berlin

2. Which language runs in a web browser?

- ☐ Java
- ☐ C
- ☒ JavaScript

3. What does HTML stand for?

- ☒ Hyper Text Markup Language
- ☐ Home Tool Markup Language
- ☐ Hyperlinks and Text Markup Language

Submit Exam

Your exam has been submitted. Thank you!

Explanation:

This HTML document represents an online exam interface with a timer and multiple-choice questions. The structure includes

a main container with a title, a timer, a form containing the questions, and a submission message. The JavaScript handles the timer countdown, form submission, and prevents multiple submissions.